

LINQ / ARCHITECTURE AND SYSTEM DESIGN

Real Backend for NGN Stablecoin Merchant Payments

A full-stack payment system where merchants onboard with business and bank details, keep control of Dynamic-managed wallets, create NGN payment links, and settle stablecoin payments through Paycrest.

FRONTEND

Next.js App Router

BACKEND

Route Handlers

DATABASE

Koyeb Postgres +
SQL

PROVIDERS

Paycrest + Dynamic

The implementation is env-gated. Production credentials activate live Paycrest, Redis, Dynamic, and Koyeb Postgres paths. Local development uses deterministic fixtures so the product can be reviewed without real-money provider access.

Generated from the current Linq worktree. Report style follows the dark, restrained gradient reference in DESIGN (1).md.

01 / EXECUTIVE OVERVIEW

What Linq Does

Linq lets a Nigerian merchant accept USDT or USDC while thinking in NGN. The merchant creates open, fixed, or static receive surfaces. Customers provide name and email, select token and network, and receive exact payment instructions.

The key separation is custody. Dynamic manages merchant wallet control. Linq stores public wallet routing metadata only. Paycrest creates settlement orders and provides receive addresses for payment/request links.

- NGN-first payment links and dashboard totals.
- Paycrest rate refresh at payment time.
- Payer details stored against each order.
- 15-minute cache for Paycrest tokens and rates.
- Static wallet mode separated from Paycrest order mode.
- Retry transfer uses the verified merchant bank account only.

Implementation Status

Implemented

Plain SQL schema, server modules, API routes, checkout order flow, merchant onboarding UI, order dashboard, webhook handler, cache abstraction, and report artifacts.

Env-Gated

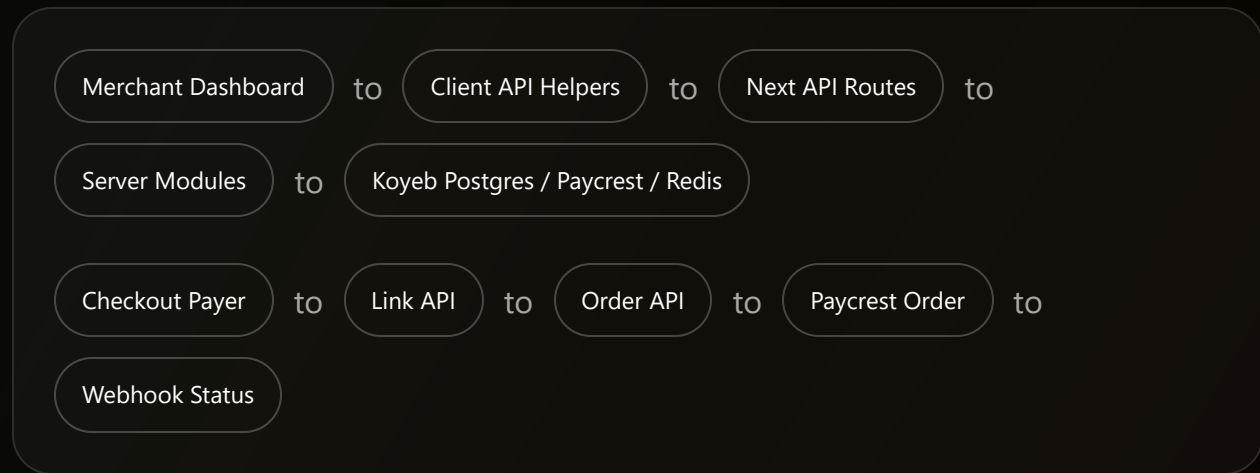
Paycrest, Redis, Koyeb Postgres, and Dynamic are activated by environment variables. Local mode keeps the app demonstrable without live credentials.

Caveat

The Dynamic SDK install hung in this environment, so the app uses a bridge provider abstraction with deterministic fallback wallets. The bridge is the intended SDK swap point.

Layered Design

The backend can now deploy separately to Koyeb from the `backend/` directory using `backend/Dockerfile`. The frontend can call the public Koyeb backend URL while the backend connects to Koyeb Postgres through `DATABASE_URL`.



Presentation Layer

Next.js pages and React components render the mobile-first dashboard, receive page, settings onboarding, and checkout. Business UI stays compact and operational.

Backend API Layer

Route handlers under `src/app/api` expose onboarding, merchant, bank, Paycrest utility, payment link, order, retry, and webhook endpoints.

Server Domain Layer

`src/server` centralizes env parsing, validation, Paycrest requests, Redis cache behavior, webhook security, API response envelopes, and store access.

Data Layer

Koyeb Postgres accessed through direct SQL and the `pg` client is the intended production store. The current local store mirrors repository behavior so flows can work without a database URL.

03 / MERCHANT ONBOARDING

Business, Bank, and Wallet Setup

- 1 Merchant signs in through Dynamic and gets merchant-controlled wallet addresses.
- 2 Settings onboarding collects business name, merchant name, business email, location, bank institution code, and account identifier.
- 3 The backend validates and sanitizes the payload with Zod.
- 4 Paycrest verifies the payout bank account through `/verify-account`.
- 5 Linq stores resolved account name, verification status, public wallet addresses, and onboarding status.

Stored Wallet Data

Wallet id, chain, network, public address, wallet type, and token support. Private keys are never requested or stored.

Stored Bank Data

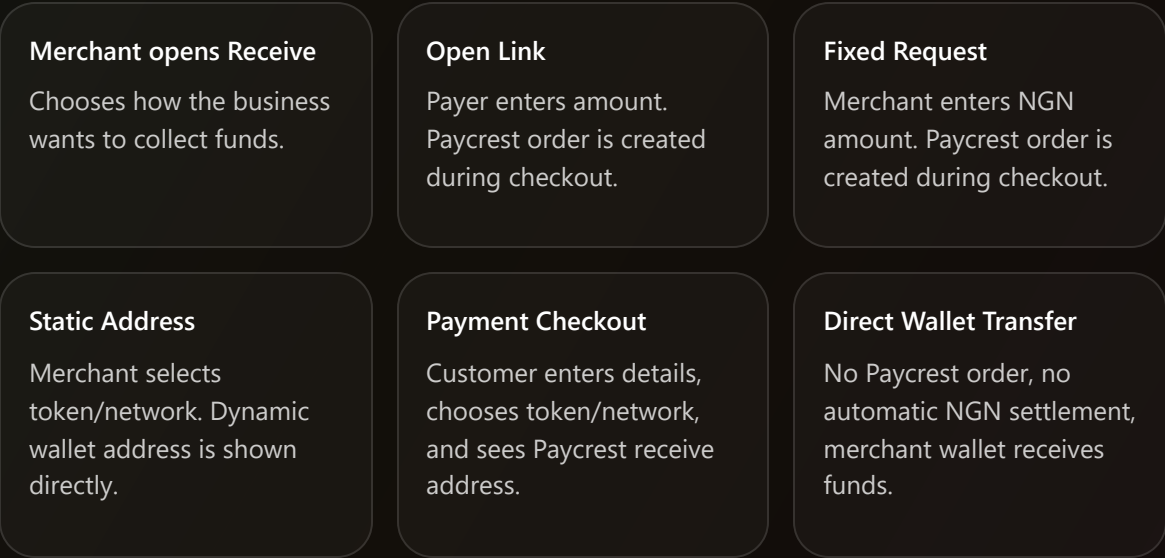
Institution code, institution name, account identifier, Paycrest-resolved account name, and verification status.

Open, Fixed, and Static Links

Mode	Amount	Paycrest Order	Address Shown	Use Case
Open	Payer enters NGN amount	Yes	Paycrest receive address	Flexible payment collection
Fixed	Merchant locks NGN amount	Yes	Paycrest receive address	Invoice or payment request
Static	No fixed settlement amount	No	Merchant Dynamic wallet address	Direct wallet receive

Static links are deliberately not Paycrest orders. This prevents the product from implying automatic NGN settlement for a transfer that goes directly to a merchant wallet.

Receive Mode Decision Diagram



Customer Payment Sequence

- 1 Checkout opens the payment link and fetches merchant/link data from `GET /api/payment-links/[id]`.
- 2 The payer's name and email are read from localStorage or collected once.
- 3 Checkout loads Paycrest-supported tokens and networks from the backend.
- 4 The payer selects USDT or USDC and a supported network.
- 5 The backend refreshes the NGN rate for the chosen token/network/amount.
- 6 The app creates a local order, then creates a Paycrest order.
- 7 Checkout displays exact crypto amount, receive address, QR code, Paycrest id, status, and expiry if provided.

Why Local Order First

Creating the local order before exposing transfer instructions gives Linq an internal audit trail even if the provider request, webhook, or customer session fails after the first attempt.

Order Flow State Diagram

Link Opened

Payer Details

Asset
SelectionNetwork
Selection

Rate Refresh

Review

Local Order

Paycrest Order

Transfer
InstructionsAwaiting
Payment

Status Polling

Final State

Happy path: pending to deposited to validated to settling to settled.

Refund path: deposited or failed provider path to refunding to refunded.

Expiry path: pending order reaches validity window and becomes expired.

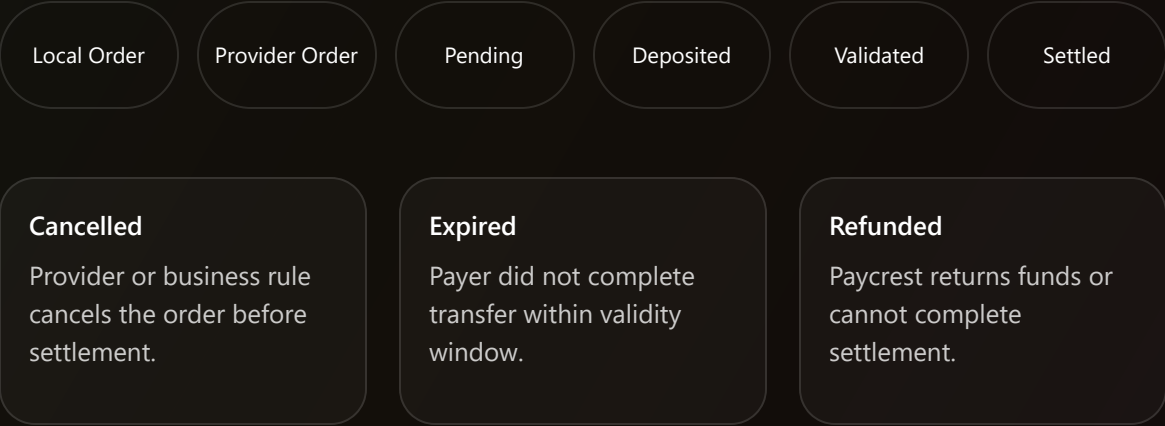
Provider Responsibilities

- Fetch supported tokens through `/tokens`.
- Fetch rates through `/rates/{network}/{token}/{amount}/{fiat}`.
- Fallback to provider rate bands through `/provider/rates/{token}/{fiat}`.
- Verify bank accounts through `/verify-account`.
- Create crypto-to-fiat orders through `/sender/orders`.
- Normalize statuses from Paycrest into internal order states.

```
{
  amount: "50000",
  amountIn: "fiat",
  source: {
    type: "crypto",
    currency: "USDT",
    network: "base",
    refundAddress: "merchant_wallet"
  },
  destination: {
    type: "fiat",
    currency: "NGN",
    recipient: {
      institution: "GTBINGLA",
      accountIdIdentifier: "1234567890"
    }
  }
}
```

Returned Paycrest values such as amount, rate, sender fee, transaction fee, receive address, and validity window become the stored source of truth for the order.

Paycrest Lifecycle Diagram



Production Schema

Model	Purpose	Important Fields
User	Maps Dynamic identity to Linq ownership.	dynamicUserId, email, name, role
MerchantBusiness	Business profile and onboarding state.	businessName, merchantName, email, location, onboardingStatus
MerchantWallet	Public wallet routing metadata.	chain, network, address, walletId, walletType, tokenSupport
BankAccount	Verified NGN payout destination.	institutionCode, accountIdentifier, resolvedAccountName, verificationStatus
PaymentLink	Open, fixed, or static checkout surface.	slug, mode, amountNgn, description, status
Order	Central payment record.	payerName, payerEmail, amountNgn, token, network, quotedRate, cryptoAmountDue, paycrestOrderId, status
OrderEvent	Audit trail for app and provider events.	source, eventName, rawPayload, createdAt
TransferAttempt	Payout retry tracking.	attemptNumber, status, paycrestReference, resultPayload, errorMessage

07A / REFUNDS AND RETRY TRANSFERS

Refund and Recovery Flow

Refund Sequence Diagram

Paycrest Status

Paycrest sends refunding, refunded, failed, cancelled, or expired status.

Webhook Handler

Linq verifies signature, stores raw event, and normalizes the status.

Order Update

The order record changes state and becomes visible in dashboard polling.

Merchant Review

Merchant expands the compact order row to inspect payer, token, rate, and provider details.

Retry Eligibility

Retry is only shown for failed, refunded, expired, or cancelled statuses.

Verified Bank Only

Retry creates a TransferAttempt using the already verified payout account.

Why Retry Is Restricted

A payout retry can move money, so it must not let a merchant swap account numbers during a failure. The retry action always reuses the stored verified bank account to reduce fraud and support operational review.

Audit Trail

Every webhook creates an OrderEvent. Every retry creates a TransferAttempt with attempt number, status, reference, payload, and error message when applicable.

08 / API SURFACE

Backend Endpoints

Endpoint	Purpose	Notes
POST /api/onboarding/merchant	Create/update merchant setup.	Verifies bank and stores wallets.
GET /api/merchant/me	Return active merchant dashboard data.	Production should bind to Dynamic auth.
POST /api/bank/verify	Verify bank correctness.	Rate limited.
GET /api/paycrest/tokens	List supported Paycrest tokens/networks.	Cached 15 minutes.
GET /api/paycrest/rate	Fetch current NGN rate.	Token/network/amount-aware.
POST /api/payment-links	Create open/fixed/static link.	Fixed links require amount.
GET /api/payment-links/[id]	Fetch public checkout data.	Returns link and merchant.
POST /api/orders	Create local and Paycrest order.	Rejects static links.
GET /api/orders	List merchant orders.	Includes transfer attempts.
GET /api/orders/[id]	Fetch one order.	Used by polling.
POST /api/orders/[id]/retry-transfer	Retry payout transfer.	Eligible terminal statuses only.
POST /api/webhooks/paycrest	Receive Paycrest status events.	HMAC verification when configured.

OWASP-Aligned Controls

Validation

All write routes parse request data through Zod. Schemas trim text, normalize email, restrict token and network values, bound amount size, and limit field lengths.

Webhook Integrity

Paycrest webhooks use HMAC SHA-256 verification with timing-safe comparison when a webhook secret is configured.

Rate Limiting

Bank verification and public order creation have rate limits. Production should move this to Redis or edge middleware.

Secrets

Paycrest API keys, webhook secrets, database URLs, Redis tokens, and Dynamic server tokens stay in environment variables and never in client bundles.

Custody

Linq stores public wallet addresses only. It never requests or stores private keys, seed phrases, MPC recovery shares, or Dynamic recovery secrets.

Authorization

Production should bind merchant routes to server-verified Dynamic sessions so one merchant cannot access another merchant's links, orders, or bank records.

Deployment and Configuration

Variable	Purpose
DATABASE_URL	Koyeb/Postgres connection string.
DATABASE_SSL	Keep true for Koyeb database TLS unless explicitly disabled.
DATABASE_POOL_MAX	Maximum direct pg pool size, default 5.
PAYCREST_API_KEY	Paycrest API authentication.
PAYCREST_API_URL	Paycrest base URL, defaults to v2.
PAYCREST_WEBHOOK_SECRET	Webhook HMAC verification.
NEXT_PUBLIC_DYNAMIC_ENV_ID	Dynamic frontend environment id.
DYNAMIC_API_TOKEN	Server-side Dynamic token if needed.
UPSTASH_REDIS_REST_URL	Redis cache URL.
UPSTASH_REDIS_REST_TOKEN	Redis cache token.
NEXT_PUBLIC_APP_URL	Public URL for generated links.
RESEND_API_KEY	Resend API key for receipt and invoice emails.
EMAIL_FROM	Verified sender identity for receipt email delivery, using Linq <noreply@useling.site> in production.
DYNAMIC_WEBHOOK_SECRET	Optional HMAC secret for Dynamic wallet incoming webhooks.

Deployment Steps

1. Provision or attach Koyeb Postgres and Redis.
2. Add all environment variables.
3. Run `db/schema.sql` against the Koyeb database.
4. Configure Paycrest webhook URL and secret.
5. Configure Dynamic app origins.
6. Deploy the Next.js app.

7. Run a tiny live Paycrest smoke test.

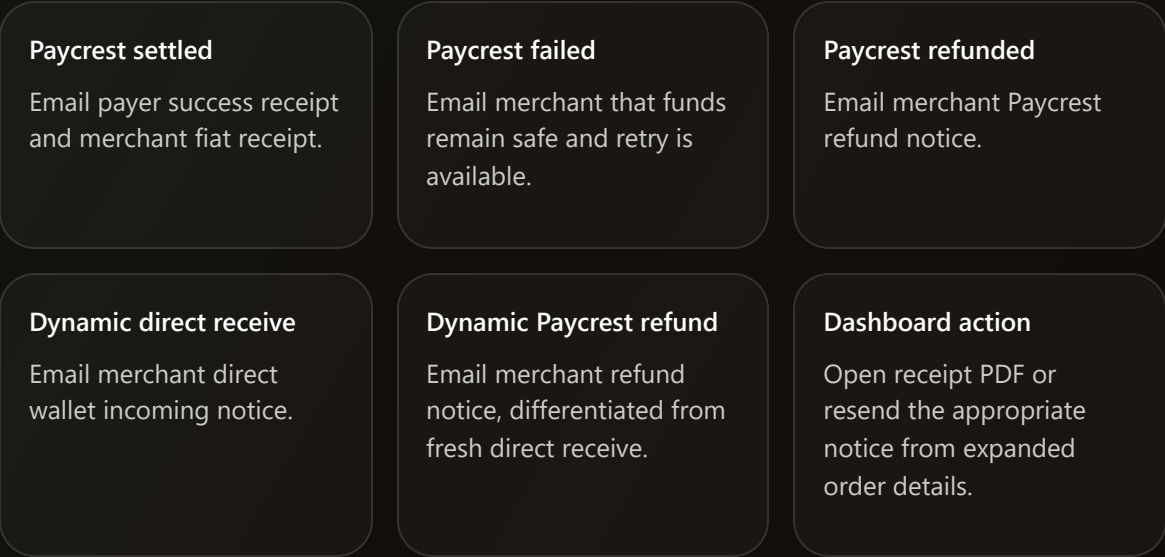
Invoice, Receipt, and Notice Generation

Linq now creates branded HTML and PDF receipts for payer success, merchant fiat settlement, payout failure, Paycrest refunds, and direct merchant wallet incoming funds. The same renderer powers downloadable PDFs and Resend email attachments.

When Resend is not configured, delivery is safely recorded as skipped so development does not accidentally send email.

Payer	Transaction success receipt
Merchant	Fiat received settlement receipt
Merchant	Payout failed, funds safe notice
Merchant	Paycrest refund notice
Merchant	Dynamic wallet incoming notice

Receipt Notification Flow



Receipt Endpoints

`GET /api/orders/[id]/receipt`, `GET /api/orders/[id]/receipt.pdf`, and `POST /api/orders/[id]/send-receipt`.

Verification Strategy

Automated Tests to Add

- Validation schemas for onboarding, bank, links, orders, and retries.
- Paycrest payload construction and status normalization.
- Rate conversion and crypto amount calculation.
- Webhook signature verification and event persistence.
- Retry transfer eligibility rules.

Manual Smoke Script

1. Submit merchant onboarding.
2. Create fixed 50,000 NGN payment request.
3. Open checkout preview.
4. Enter payer name and email.
5. Select USDT and Base.
6. Create manual transfer order.
7. Confirm QR, address, order id, and status.
8. Open dashboard orders and expand the new order.

Known Next Steps

Install real Dynamic SDK packages, replace the local store with direct SQL repository functions through `src/server/db.ts`, enforce server-side Dynamic authorization, move rate limits to Redis, add webhook idempotency keys, and add full automated test coverage.

Current evidence: `npm run build` passed after the backend routes, Koyeb SQL schema, checkout flow, dashboard, and report artifacts were added.